Gestión de clientes Tienda Deportiva JR

PHP nativo · Sesiones · MySQL/MariaDB

Ampliación del proyecto de JOHANSTIVENJR12 con registro seguro, inicio y cierre de sesión, panel Mi Cuenta, direcciones y pedidos vinculados al usuario. El checkout permite comprar con cuenta o como invitado.

Punto de partida. Se revisaron las ramas master y maain. El commit base c8e2905 contiene una tienda estática con seis productos y recursos Bootstrap. No había tablas, scripts PHP, carrito ni checkout. Por eso se han añadido esas piezas y se han conservado los productos, las fotografías y la identidad visual.

Elemento	Resultado
Código	Rama local feature/login desde master. PHP nativo sin frameworks de servidor.
Base de datos	Volcado SQL real con estructura y datos ficticios; reimportación comprobada.
Memoria	Explicación de decisiones, código, capturas reales y pruebas.
Publicación en GitHub	Pendiente de acceso de escritura y del usuario del profesor. No hay PR publicada.

Repositorio de origen:
github.com/JOHANSTIVENJR12/tienda_deportivaJR

Documento elaborado con asistencia de Codex. Las pruebas descritas se ejecutaron en este entorno. La revisión personal del estudiante y la asignación del profesor como reviewer siguen siendo necesarias antes de presentar la práctica.

1. Preparación y estructura

Orden de implementación

1. Revisar las ramas y localizar el catálogo, imágenes y estilos.
2. Crear feature/login desde master y definir conexión, sesiones y funciones compartidas.
3. Implementar las tablas y los flujos de registro, login y logout.
4. Añadir Mi Cuenta, carrito y checkout vinculados al usuario.
5. Probar en navegador y base de datos, exportar el SQL y preparar la documentación.

Cómo arrancar

Se requiere PHP 8.2+ con pdo_mysql y mbstring, MySQL 8.0.16+ o MariaDB 10.4+, y un servidor PHP. En XAMPP: copiar la tienda a htdocs, iniciar Apache y MySQL e importar database/tienda_deportiva_jr.sql en una base nueva. Copiar config/local.example.php como config/local.php y ajustar las credenciales. Abrir la tienda por HTTP, nunca mediante file:// ni Live Server.

La comprobación se realizó con PHP 8.2.12, MariaDB 10.4.32 y Chrome. Se creó una instancia de base de datos independiente en el puerto 3307; la configuración de ejemplo para XAMPP usa el puerto habitual 3306. config/local.php queda excluido de Git.

Ubicación	Responsabilidad
config/	PDO y configuración local sin publicar credenciales.
includes/	Sesión, CSRF, validación, escape, catálogo y vistas compartidas.
Scripts PHP de la raíz	Procesan el formulario antes de imprimir HTML; redirecciones HTTP 303.
database/	Esquema, semilla y exportación completa.
tests/ y docs/	Pruebas reproducibles, resultados, capturas, PDF y descripción de PR.

Se separan responsabilidades sin introducir un framework: las páginas usan las mismas funciones y la misma navegación. Los recursos Bootstrap locales permiten ejecutar la tienda sin depender de un CDN.

2. Modelo de datos

Relaciones: usuarios 1:N direcciones; usuarios 1:N pedidos; pedidos 1:N pedido_lineas. Un pedido de invitado no tiene usuario asociado. Todas las tablas usan InnoDB y utf8mb4.

CONSULTAS REALES · MARIADB 10.4.32

Base de datos · Datos ficticios iniciales

SELECT ejecutados mediante PDO sobre tienda_deportiva_jr. Las contraseñas se almacenan como hashes.

Usuarios

id	nombre	email	password
1	Ana Demo	ana@example.com	$2y$10$… (60 caracteres)
2	Bruno Demo	bruno@example.com	$2y$10$… (60 caracteres)

Direcciones

usuario_id	calle	ciudad	cp	tipo	principal
1	Calle Ejemplo 10, 2 A	Madrid	28001	entrega	1
1	Avenida Prueba 25	Madrid	28002	facturación	1
2	Calle Otra Cuenta 8	Valencia	46001	entrega	1

Pedidos

id	usuario_id	nombre	total	estado
1	1	Ana Demo	45.00	entregado
2	2	Bruno Demo	120.00	pendiente
3	NULL	Invitado Demo	8.00	pendiente

pedido_lineas: 4 filas. Volcado reimportado: 2 usuarios · 3 direcciones · 3 pedidos · 4 líneas.

Figura 1. Vista de consultas SELECT reales mediante PDO. Los datos son ficticios; se muestra solo el prefijo del hash para facilitar la lectura.

El email es único. La contraseña dispone de VARCHAR(255) para admitir futuros algoritmos de hash. El CP se guarda como CHAR(5) para conservar ceros iniciales; tipo distingue entrega y facturación mediante ENUM. Los campos de dirección se validan también en PHP.

usuario_id en pedidos admite NULL; la clave foránea usa ON DELETE SET NULL para conservar compras históricas. Las direcciones usan ON DELETE CASCADE. Los importes se almacenan en DECIMAL(10,2); los cálculos PHP se realizan en céntimos enteros. Se guardan nombre, precio y dirección en el pedido para que no dependan de cambios futuros en el catálogo o en la agenda.

El volcado completo se generó con mysqldump de MariaDB y se reimportó en otra base vacía: 2 usuarios, 3 direcciones, 3 pedidos y 4 líneas. No se necesita ALTER TABLE porque el proyecto de origen no tenía esquema previo.

3. Registro de clientes

registro.php valida nombre, email y contraseña en el servidor. Exige confirmación de contraseña y una longitud de 8 a 72 bytes; el límite superior evita que bcrypt trunque silenciosamente la entrada en PHP 8.2. No se recorta ni modifica la contraseña.

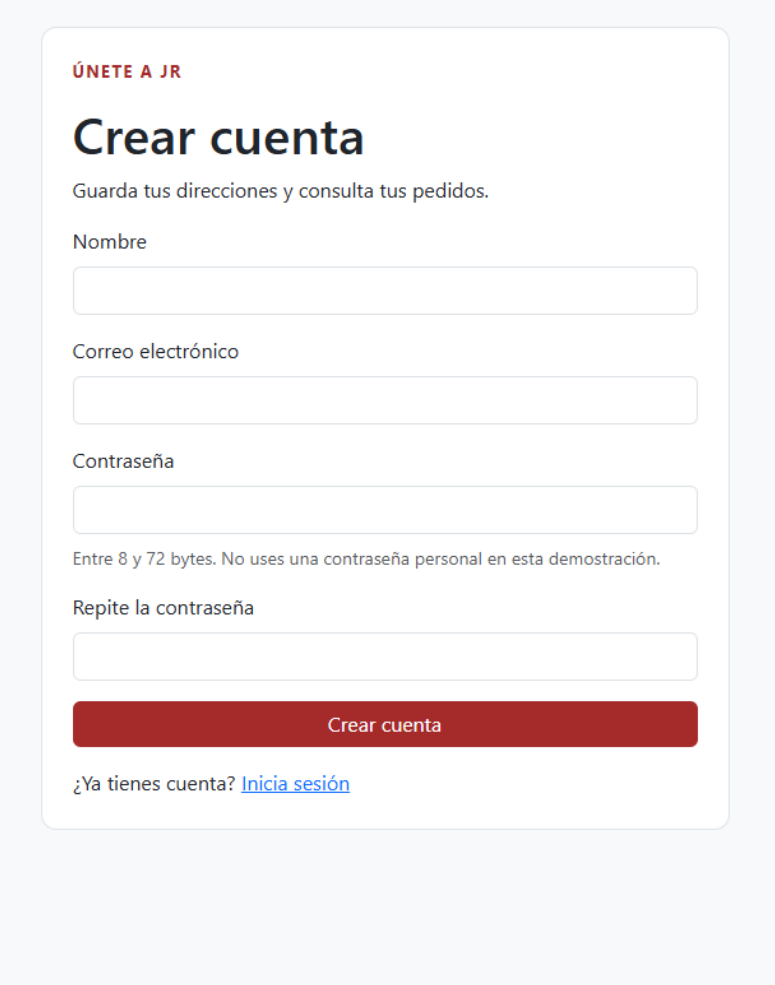


Figura 2. Formulario real de registro antes de enviar los datos.

Se almacena password_hash(password, PASSWORD_DEFAULT) mediante una consulta preparada. Los marcadores separan los valores del SQL. La restricción UNIQUE de email resuelve también altas simultáneas; el error 1062 se transforma en un mensaje comprensible. Otros errores no se confunden con un correo duplicado.

Después del alta se redirige al login y se muestra una confirmación. El navegador no conserva los campos de contraseña. La prueba automatizada creó una cuenta y comprobó que un segundo registro con el mismo correo era rechazado.

4. Login y protección de sesión

login.php busca el correo con una consulta preparada y compara la contraseña con password_verify(). Al acertar, regenera el identificador de sesión y guarda usuario_id y nombre en $_SESSION. Si el algoritmo lo requiere, actualiza el hash con password_needs_rehash().

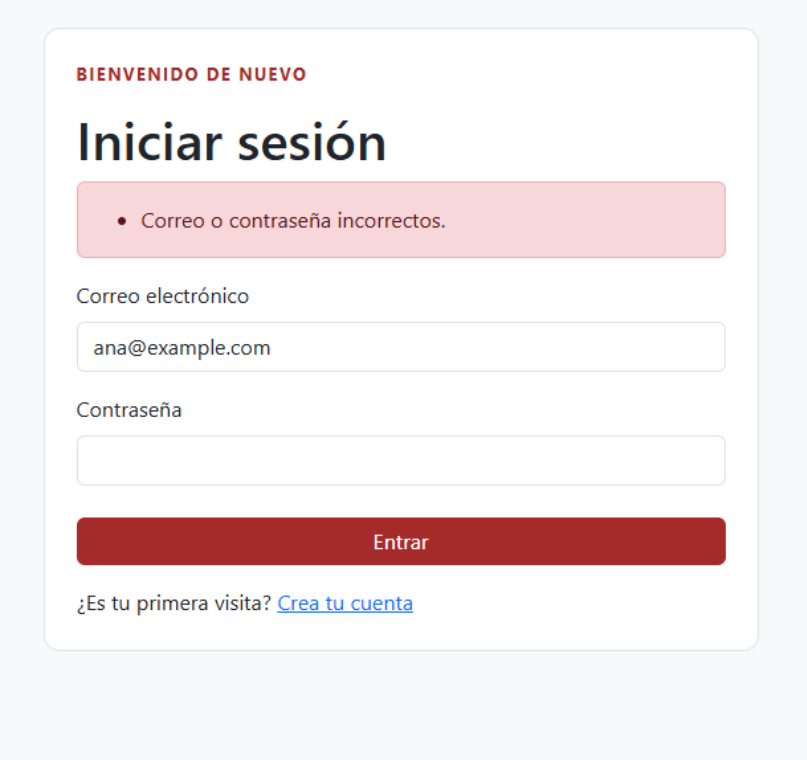


Figura 3. Credenciales incorrectas: el usuario permanece en login y recibe un mensaje genérico.

mi-cuenta.php invoca require_login() antes de emitir HTML. Sin usuario_id en sesión, responde con una redirección a login.php. La prueba de acceso protegido terminó en esa página. También se compararon las cookies antes y después del login para verificar el cambio del identificador.

Logout solo acepta POST con CSRF válido, vacía $_SESSION, caduca la cookie y destruye la sesión. Tras cerrar sesión, volver a Mi Cuenta redirige otra vez al login. La cookie usa HttpOnly y SameSite=Lax; Secure se activa cuando la conexión es HTTPS. Cache-Control: no-store evita almacenar estas páginas como contenido reutilizable.

El enlace desde checkout admite únicamente el destino interno checkout. No se acepta una URL arbitraria como destino de redirección.

5. Mi Cuenta y direcciones

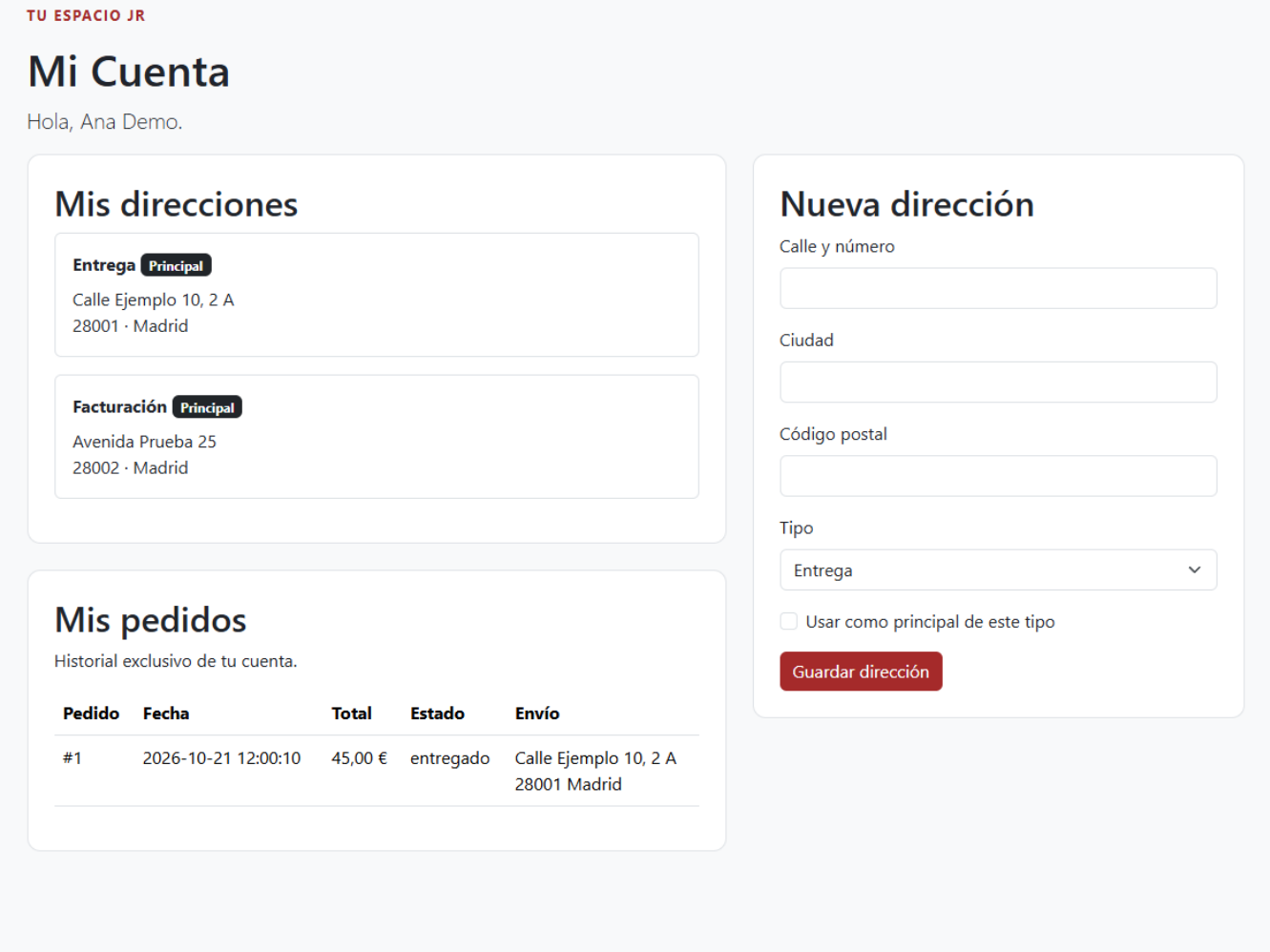


Figura 4. Ana ve sus direcciones de entrega y facturación y su pedido inicial. No aparecen los datos de Bruno.

El formulario permite guardar calle, ciudad, CP y tipo. La primera dirección de cada tipo se marca como principal. Al crear otra, la casilla permite sustituir la principal de ese tipo. La dirección de facturación no se utiliza para rellenar la entrega.

El alta se ejecuta en una transacción. SELECT ... FOR UPDATE bloquea la fila del usuario mientras se desmarca la principal anterior y se inserta la nueva. Así, dos altas concurrentes del mismo usuario a través de esta aplicación se serializan.

Las direcciones y los pedidos se consultan con WHERE usuario_id = ? y el valor procede de la sesión. No se confía en un usuario_id enviado desde el formulario. Se probó el aislamiento iniciando sesión como Ana y como Bruno en momentos distintos. Todos los datos se imprimen con htmlspecialchars().

6. Carrito y checkout con cuenta

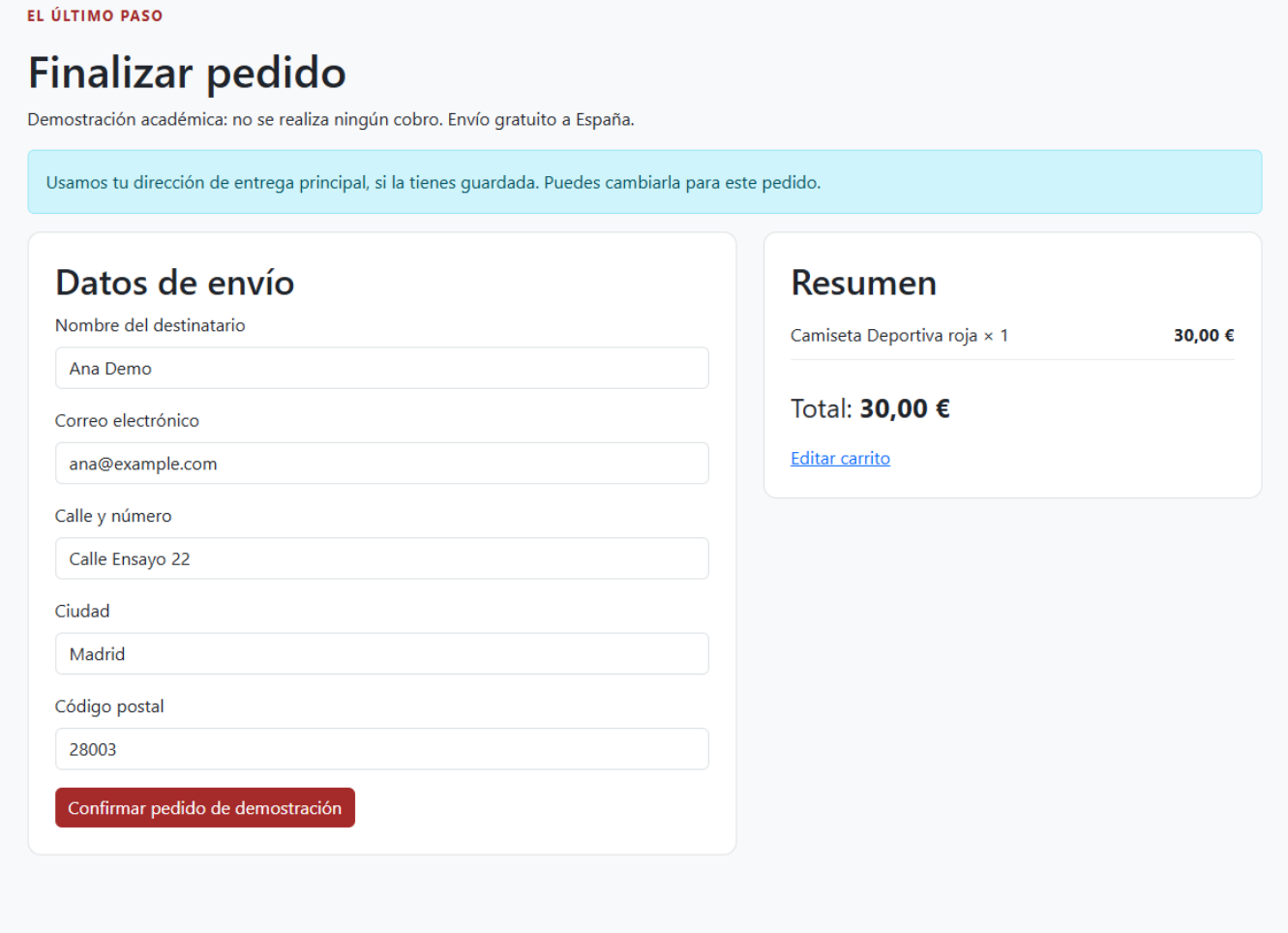


Figura 5. La nueva dirección principal Calle Ensayo 22 aparece automáticamente en el checkout.

El carrito guarda en sesión identificadores y cantidades de 1 a 20. Los precios se obtienen del catálogo PHP del servidor. El cliente no envía el total que se debe guardar. El carrito permite añadir, actualizar y eliminar productos.

Al abrir checkout.php con sesión, se consultan nombre y email del usuario y su dirección de entrega. La consulta ordena principal DESC y usa LIMIT 1. Si no hay dirección guardada, esos campos permanecen vacíos. Los valores se insertan escapados en los atributos value del formulario.

El usuario puede editar la dirección para esa compra sin modificar la agenda. Si falla una validación del POST, se conservan los datos que acaba de introducir. Si se confirma correctamente, se guardan el pedido y todas sus líneas dentro de una única transacción.

7. Compra como invitado

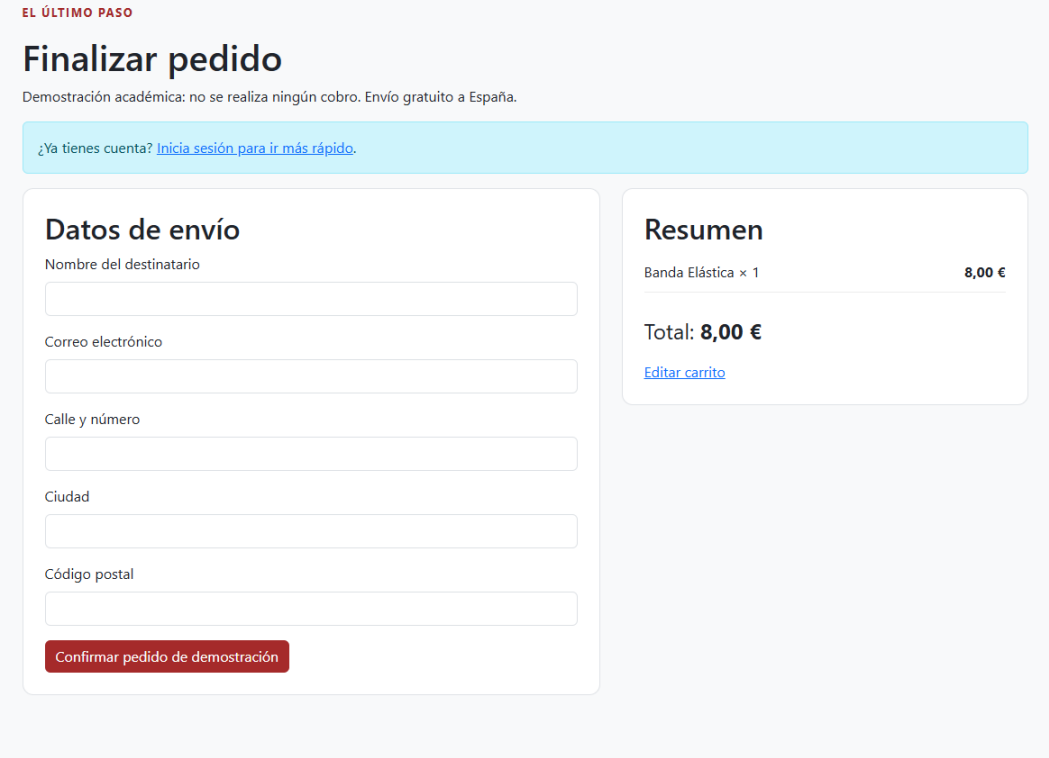


Figura 6. Checkout sin sesión: campos vacíos y enlace para iniciar sesión.

El invitado puede completar el mismo formulario. En el INSERT, uid() devuelve NULL; no se asocia el pedido a una cuenta por coincidencia de email. Tener el mismo correo que un cliente no concede acceso a sus pedidos.



Figura 7. Confirmación real de un pedido de prueba. No existe ningún cobro.

Tras confirmar, se vacía el carrito y se redirige a pedido-confirmado.php. Esa página no consulta pedidos por un ID recibido del navegador: usa el último identificador guardado en la propia sesión. Un token único de checkout y la redirección tras el POST ayudan a impedir reenvíos duplicados.

8. Código auditado: registro

CÓDIGO REAL DEL PROYECTO

registro.php · líneas 7-27

Vista generada a partir del archivo guardado. Los números corresponden al código fuente.

```
 7      $nombre = post('nombre'); $email = strtolower(post('email'));
 8      $password = is_string($_POST['password'] ?? null) ? $_POST['password'] : '';
 9      if ($nombre === '' || mb_strlen($nombre) > 100) $errors[] = 'El nombre es obligatorio y debe tener
como máximo 100 caracteres.';
10      if (!filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($email) > 190) $errors[] = 'Introduce un
correo electrónico válido.';
11      // PASSWORD_DEFAULT usa bcrypt en PHP 8.2: no aceptar truncamientos a 72 bytes.
12      if (strlen($password) < 8 || strlen($password) > 72 || str_contains($password, "\0")) $errors[] = 'La
contraseña debe ocupar entre 8 y 72 bytes.';
13      if ($password !== ($_POST['confirmacion'] ?? null)) $errors[] = 'Las contraseñas no coinciden.';
14      if (!$errors) {
15          try {
16              $stmt = db()->prepare('INSERT INTO usuarios (nombre, email, password) VALUES (?, ?, ?)');
17              $stmt->execute([$nombre, $email, password_hash($password, PASSWORD_DEFAULT)]);
18              flash('Cuenta creada. Ya puedes iniciar sesión.');
19              redirect('login.php');
20          } catch (PDOException $error) {
21              if (($error->errorInfo[1] ?? null) !== 1062) throw $error;
22              $errors[] = 'Ese correo ya tiene una cuenta. Inicia sesión.';
23          }
24      }
25  }
26  $title = 'Crear cuenta'; require __DIR__ . '/includes/header.php';
27  ?>
```

Figura 8. Fragmento real de registro.php, con numeración de líneas.

Cómo y por qué: primero se valida; solo si no hay errores se calcula el hash y se realiza el INSERT. password_hash genera una sal automáticamente. No se implementa un algoritmo propio ni se cifra de forma reversible la contraseña.

La consulta usa tres marcadores para nombre, email y hash. El email único es una garantía de base de datos, además de la validación PHP. Tras crear la cuenta se guarda un mensaje temporal y se redirige con 303 para que recargar la página no vuelva a registrar al usuario.

9. Código auditado: login

CÓDIGO REAL DEL PROYECTO

login.php · líneas 6-24

Vista generada a partir del archivo guardado. Los números corresponden al código fuente.

```
 6  $next = ($_GET['next'] ?? '') === 'checkout' ? 'checkout.php' : 'mi-cuenta.php';
 7  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
 8      check_csrf(); $email = strtolower(post('email'));
 9      $password = is_string($_POST['password'] ?? null) ? $_POST['password'] : '';
10      $stmt = db()->prepare('SELECT id, nombre, password FROM usuarios WHERE email = ?');
11      $stmt->execute([$email]); $user = $stmt->fetch();
12      if ($user && password_verify($password, $user['password'])) {
13          if (password_needs_rehash($user['password'], PASSWORD_DEFAULT)) {
14              db()->prepare('UPDATE usuarios SET password = ? WHERE id = ?')-
>execute([password_hash($password, PASSWORD_DEFAULT), $user['id']]);
15          }
16          session_regenerate_id(true);
17          $_SESSION['usuario_id'] = (int) $user['id'];
18          $_SESSION['nombre'] = $user['nombre'];
19          unset($_SESSION['csrf'], $_SESSION['checkout_token']);
20          redirect($next);
21      }
22      $errors[] = 'Correo o contraseña incorrectos.';
23  }
24  $title = 'Iniciar sesión'; require __DIR__ . '/includes/header.php';
```

Figura 9. Fragmento real de login.php: SELECT, verificación, posible rehash y renovación de sesión.

Se consulta únicamente la cuenta que coincide con el correo. password_verify reconoce el algoritmo y la sal incluidos en el hash. No se comparan contraseñas en SQL ni se vuelve a hashear el texto para intentar igualar dos hashes aleatorios.

session_regenerate_id(true) reduce el riesgo de fijación de sesión. El ID del usuario se toma de la fila devuelta por la base, no de un campo del formulario. Se renuevan los tokens de formularios al entrar. Las credenciales incorrectas reciben un mensaje genérico sin revelar qué parte falló.

10. Código auditado: datos propios

CÓDIGO REAL DEL PROYECTO

mi-cuenta.php · líneas 27-36

Vista generada a partir del archivo guardado. Los números corresponden al código fuente.

```
 27  $stmt->execute([uid()]); $addresses = $stmt->fetchAll();
 28  // El filtro procede de la sesión, nunca de un parámetro enviado por el cliente.
 29  $stmt = db()->prepare('SELECT id, total, estado, fecha, calle, ciudad, cp FROM pedidos WHERE usuario_id =
? ORDER BY fecha DESC, id DESC');
 30  $stmt->execute([uid()]); $orders = $stmt->fetchAll();
 31  $title = 'Mi Cuenta'; require __DIR__ . '/includes/header.php';
 32  ?>
 33  <p class="eyebrow">Tu espacio JR</p><h1>Mi Cuenta</h1><p class="lead">Hola, <?= e($_SESSION['nombre']) ?
>.</p>
 34  <div class="row"><section class="col-lg-7"><div class="panel"><h2>Mis direcciones</h2>
 35  <?php if (!$addresses): ?><p>Aún no has guardado ninguna dirección.</p><?php endif; ?>
 36  <?php foreach ($addresses as $address): ?><div class="border rounded p-3 mb-3"><strong><?=
e(ucfirst($address['tipo'])) ?></strong> <?php if ($address['principal']): ?><span class="badge bg-
dark">Principal</span><?php endif; ?><p class="mb-0 mt-2"><?= e($address['calle']) ?><br><?= e($address['cp'])
?> · <?= e($address['ciudad']) ?></p></div><?php endforeach; ?></div>
```

Figura 10. Consultas reales del panel. Ambos listados filtran por el usuario de la sesión.

El WHERE usuario_id = ? es el control que mantiene separados los datos de cada cliente. Proteger la página con sesión es necesario, pero además cada consulta debe limitar las filas que devuelve.

CÓDIGO REAL DEL PROYECTO

checkout.php · líneas 3-14

Vista generada a partir del archivo guardado. Los números corresponden al código fuente.

```
  3  if (!($_SESSION['carrito'] ?? [])) { flash('Añade productos antes de tramitar un pedido.');
redirect('carrito.php'); }
  4  $errors = []; $data = ['nombre' => '', 'email' => '', 'calle' => '', 'ciudad' => '', 'cp' => ''];
  5  if (uid()) {
  6      $stmt = db()->prepare('SELECT nombre, email FROM usuarios WHERE id = ?'); $stmt->execute([uid()]);
  7      $data = array_merge($data, $stmt->fetch() ?: []);
  8      $stmt = db()->prepare("SELECT calle, ciudad, cp FROM direcciones WHERE usuario_id = ? AND tipo =
'entrega' ORDER BY principal DESC, id ASC LIMIT 1");
  9      $stmt->execute([uid()]); $data = array_merge($data, $stmt->fetch() ?: []);
 10  }
 11  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
 12      check_csrf();
 13      foreach ($data as $key => $value) $data[$key] = post($key);
 14      $errors = address_errors($data);
```

Figura 11. Consulta de usuario y dirección para el autocompletado. Se distingue explícitamente el tipo entrega.

El orden de la consulta prioriza la dirección principal. Si se devuelve false porque no hay filas, se combina un array vacío con los valores por defecto. Así no se intenta leer una dirección inexistente.

11. Código auditado: pedido atómico

CÓDIGO REAL DEL PROYECTO

checkout.php · líneas 22-39

Vista generada a partir del archivo guardado. Los números corresponden al código fuente.

```
 22              $stmt = $pdo->prepare('INSERT INTO pedidos (usuario_id, nombre, email, calle, ciudad, cp,
total, token) VALUES (?, ?, ?, ?, ?, ?, ?, ?)');
 23              $stmt->execute([uid(), $data['nombre'], $data['email'], $data['calle'], $data['ciudad'],
$data['cp'], number_format(cart_total() / 100, 2, '.', ''), $_SESSION['checkout_token']]);
 24              $orderId = (int) $pdo->lastInsertId();
 25              $stmt = $pdo->prepare('INSERT INTO pedido_lineas (pedido_id, producto_id, producto,
precio_unitario, cantidad) VALUES (?, ?, ?, ?, ?)');
 26              foreach ($_SESSION['carrito'] as $id => $quantity) {
 27                  $product = products()[$id];
 28                  $stmt->execute([$orderId, $id, $product['nombre'], number_format($product['precio'] /
100, 2, '.', ''), $quantity]);
 29              }
 30              $pdo->commit();
 31          } catch (Throwable $error) { if ($pdo->inTransaction()) $pdo->rollBack(); throw $error; }
 32          $_SESSION['ultimo_pedido'] = $orderId;
 33          unset($_SESSION['carrito'], $_SESSION['checkout_token']);
 34          redirect('pedido-confirmado.php');
 35      }
 36  }
 37  $_SESSION['checkout_token'] ??= bin2hex(random_bytes(32));
 38  $title = 'Checkout'; require __DIR__ . '/includes/header.php'; ?>
 39  <p class="eyebrow">El último paso</p><h1>Finalizar pedido</h1><p>Demostración académica: no se realiza
ningún cobro. Envío gratuito a España.</p>
```

Figura 12. Inserción del pedido y sus líneas dentro de beginTransaction/commit, con rollback si ocurre un error.

Un pedido sin sus líneas dejaría información incoherente. La transacción agrupa las inserciones: o se confirman todas o se revierte el conjunto. El identificador recién generado se obtiene con lastInsertId() en la misma conexión PDO.

Cada línea conserva producto, precio y cantidad. Los importes se formatean como decimales al guardarlos; los cálculos parten de céntimos enteros. La suma de las líneas se contrastó con el total del pedido mediante una consulta SQL.

El token del checkout tiene índice UNIQUE. El servidor mantiene bloqueada la sesión durante la petición y elimina el carrito y el token tras confirmar la transacción. No se habilita una pasarela de pago real.

12. Verificación y correcciones

Se ejecutaron **24 comprobaciones de navegador** con Playwright y Chrome, además de las comprobaciones PDO de tests/database.php. Los resultados están guardados en docs/resultados-pruebas.json, resultados-base-datos.txt y resultado-importacion.txt.

Grupo	Evidencia obtenida
Acceso y registro	Mi Cuenta sin sesión redirige; CSRF ausente da 403; registro correcto y rechazo del duplicado.
Sesiones	Credencial incorrecta rechazada; login correcto; cambio de ID de sesión; logout vuelve a bloquear el panel.
Privacidad y direcciones	Ana y Bruno ven datos diferentes; se guarda la nueva principal; el texto HTML se escapa frente a XSS.
Pedidos	Autocompletado correcto; compra autenticada e invitada; historial actualizado; reenvío sin nuevo pedido.
Base de datos	Hashes verificables; invitados con NULL; cantidades y totales coherentes; tres claves foráneas; una principal por tipo.
Interfaz	Captura de catálogo móvil a 390 px sin desbordamiento horizontal; sin respuestas HTTP 500 durante el recorrido.

Incidencias corregidas

La primera importación del esquema desde el cliente de consola usó una codificación de conexión incorrecta: el valor facturación no coincidía con el ENUM y MariaDB devolvió el error 1265. Se recreó únicamente la base de prueba y se importaron esquema y datos con --default-character-set=utf8mb4. La importación y el checkout se verificaron después.

La descarga inicial por Git falló con el backend TLS de Windows. Se usó OpenSSL para clonar el mismo repositorio sin desactivar la verificación TLS. La publicación no pudo autenticarse: ese paso sigue pendiente y no afecta a las pruebas locales del código.

Las pruebas crean datos adicionales en una base aislada. Después se restauraron los datos iniciales para generar el SQL final. Las capturas de compra muestran datos de ese recorrido, por eso incluyen Calle Ensayo 22, que no forma parte de la semilla inicial.

13. Uso de IA y entrega en Git

Transparencia sobre la asistencia

Se utilizó Codex para inspeccionar el repositorio, escribir PHP y SQL, diseñar las validaciones, ejecutar las pruebas y redactar esta memoria. La petición inicial real fue «hazme esto por favor y bien hecho», acompañada del enunciado. Después se proporcionó el enlace al repositorio. No se inventan prompts adicionales ni una revisión personal del estudiante que todavía no haya ocurrido.

A partir del enunciado, el agente concretó el trabajo en registro con hash, login con sesión, panel protegido, consultas por usuario y checkout con dirección principal. La auditoría técnica realizada consistió en revisar esos puntos en el código, comprobar la sintaxis PHP, ejecutar el recorrido real en navegador y contrastar los datos mediante SQL. La incidencia de codificación y la falta de autenticación Git se describen en la página anterior.

Lo que debes saber explicar

Por qué password_hash y password_verify trabajan juntos; de dónde sale usuario_id; por qué cada SELECT del panel necesita su filtro; cómo se elige la principal; por qué un invitado usa NULL; para qué sirven una transacción, las claves foráneas, CSRF y htmlspecialchars. Antes de presentar, revisa estos puntos personalmente y adapta cualquier explicación a lo que realmente comprendas.

Rama y Pull Request

El desarrollo se ha realizado en la rama local **feature/login**, creada desde **master**. La copia entregada conserva el historial Git y los commits separados de implementación y documentación. El destino correcto de la PR es master, que es la rama principal real del repositorio.

Pendiente: publicar con git push -u origin feature/login desde una sesión con acceso de escritura; abrir una PR contra master usando docs/PULL_REQUEST.md; seleccionar al profesor en Reviewers. El conector actual tiene push: false y Git no pudo autenticarse. Falta conocer el usuario del profesor. No se ha abierto ni fusionado ninguna PR.

Entrega preparada: código fuente y documentación, database/tienda_deportiva_jr.sql y esta memoria PDF. El checkout es académico, sin cobros ni stock. No incluye recuperación de contraseña, verificación de email ni limitación persistente de intentos; esas funciones quedan fuera de esta práctica y serían necesarias para ampliar la tienda a un entorno real.